

Quarterly Business Review

What Drives Us

Excellence

High standards in every deliverable we ship.

Integrity

Transparent and honest in all we do.

Innovation

Investing in ideas that shape tomorrow.

By the Numbers

127%

Revenue Target

94%

Retention Rate

48 NPS

Net Promoter

Side-by-Side Evaluation

Legacy Platform

Deploy time: 4 hours

Manual scaling

On-premise; high cost

Modern Stack

Deploy time: 8 min

Auto-scales fast

Cloud-native; low cost

Key Milestones

Q1

Foundation

Cloud platform migration done.

Q2

API Launch

REST API live with 99.99% SLA.

Q3

AI Beta

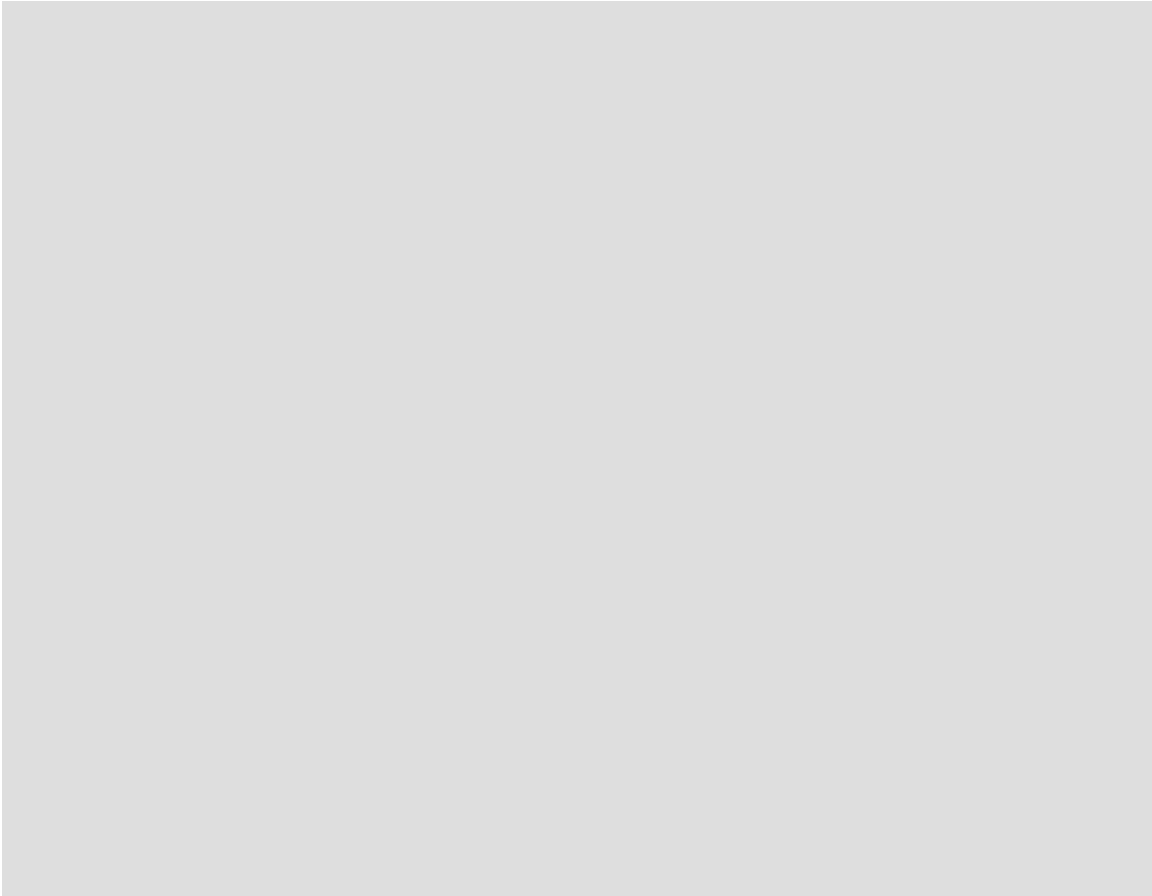
AI features shipped to 500 partners.

Q4

Scale Out

APAC regions live; global coverage.

Built in the Field



Built with customers, not just for them.

60% faster rollout; 50% fewer tickets.

What We Offer

Insights

Live data
and AI
analytics.

Security

Zero-trust,
SOC 2
ready.

Speed

Sub-100ms
p99 latency.

APIs

300+ open
connectors.

Annual Strategy Review

Executive Leadership — January 2026

2025 Results at a Glance

127%

Revenue Target

\$3.8B total

94%

Retention Rate

48 NPS

Net Promoter

+12 pts YoY

What Drove Growth

Enterprise & Products

Enterprise up 42%

APAC beat forecast 31%

New products: 18%

Core & Channel

SMB churn down

Services attach: 34→51%

Partner channel: 28%

Strategic Priorities 2026

Q1

Foundation

Merge
legacy
systems;
unified data
layer.

Q2

AI Assist

AI
workflows
GA for all
customers.

Q3

Expansion

Fortune
500 sales
motion
launched.

Q4

Open Beta

ISV
program;
target 50
partners.

Investment Areas

Build

Reliability
and dev
tooling.

Growth

Enterprise
sales and
channels.

Global

APAC and
EMEA
expansion.

Analytics

Customer
and usage
insights.

Component Gallery

All eight components in one deck

Icon Text Rows

Speed

Deploy in minutes.

Security

Safe by default.

Scale

Grows with you.

Stat Callout

99.9%

Uptime

< 50ms

Latency

p99 globally

500+

Integrations

Comparison Columns

Before

Manual; error-prone

Weekly reports

After

Automated; reliable

Real-time data

Timeline

Month 1

Kickoff

Setup and
team
training.

Month 3

Go Live

Full
production
rollout.

Month 6

Growth

Tuning and
expansion.

Image Half Bleed

Full-bleed image
with text side.



Card Grid

Feature A

Fast and
reliable.

Feature B

Secure
by
design.

Feature C

Scales
with you.

Feature D

Pay as
you go.



01

Where We Stand Today

What We Will Cover

- 01 Performance against targets
- 02 Customer and market signals
- 03 Operational health and risks
- 04 Roadmap for next quarter
- 05 Decisions we need today

3.2M

Active customers worldwide



"We stopped buying tools and started buying outcomes."

- VP of Engineering, enterprise customer

Net Revenue Retention

The percentage of recurring revenue retained from existing customers over a period, including expansion and after churn.



**What would we build if we
trusted our customers
completely?**

What Changed This Quarter

Activation time dropped by half

Support tickets fell sharply

Expansion revenue outpaced new sales



hello.py

```
# The classic, end to end  
def main():  
    print("Hello, world!")
```

main()

```
$ python hello.py  
Hello, world!
```

Three Steps to Onboard

01 Connect your data

Link a source in minutes with a guided wizard.

02 Pick a template

Start from a proven layout, then tailor it.

03 Share the result

Publish to your team and gather feedback fast.

Onboarding, Reworked

Before

Manual account setup

Three-day approval wait

Email-only support

After

One-click provisioning

Same-day approval

In-product live chat

Cloud or On-Prem

3 wks

Cloud time-to-launch

vs

6 mo

On-prem time-to-launch

Key Metrics This Quarter

98.7%

Uptime

1,240

New accounts

4.6/5

Support rating

\$2.1M

Net new revenue

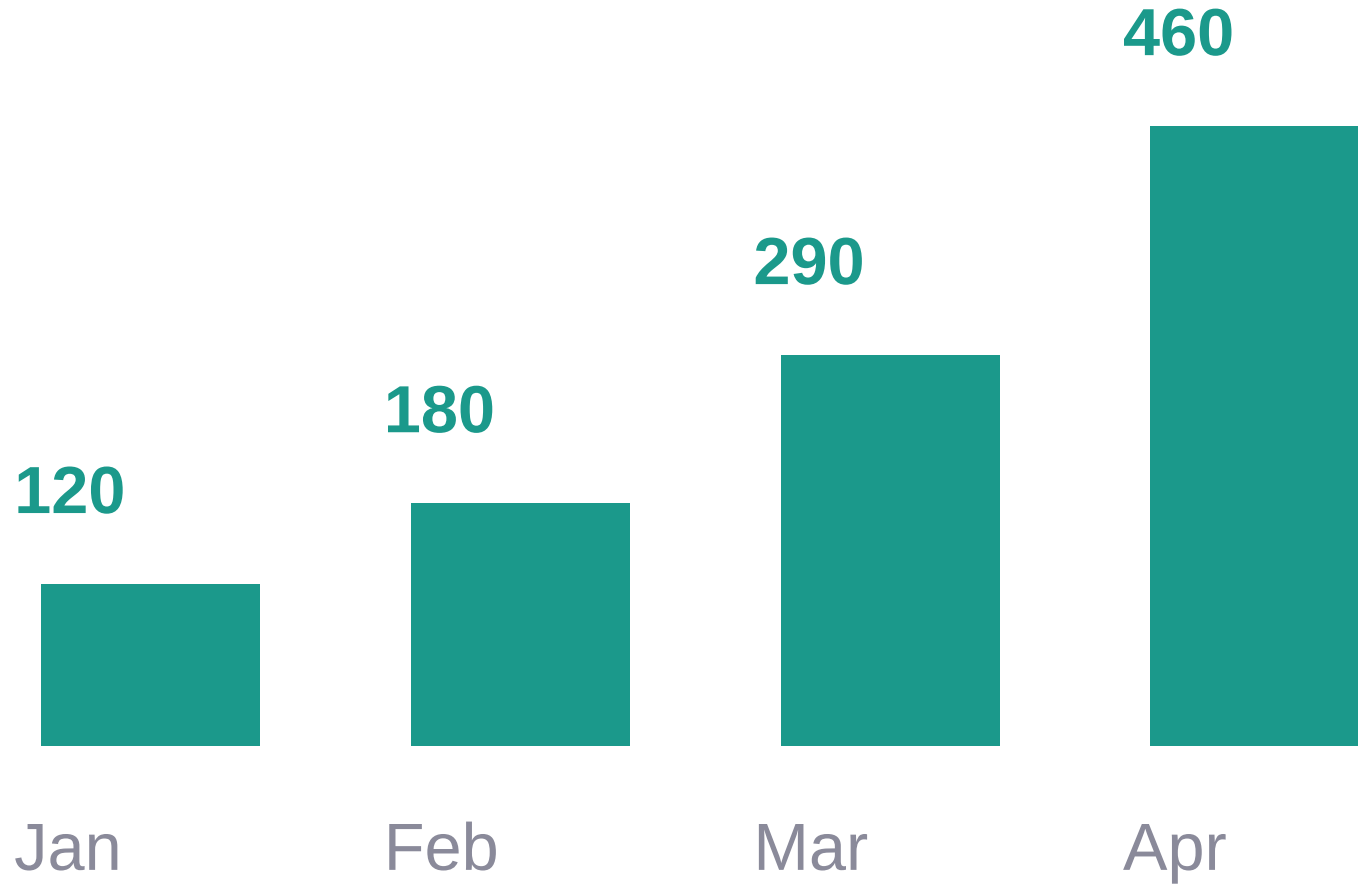
12 days

Avg. time to
value

31%

Expansion rate

Monthly Active Teams



**Activations
nearly
quadrupled
once self-serve
onboarding
shipped in
February.**

Pricing Tiers

Plan	Seats	Price
Starter	Up to 5	\$0
Team	Up to 50	\$12/seat
Business	Up to 250	\$20/seat
Enterprise	Unlimited	Custom

Performance vs Last Year

\$8.4M

Annual recurring
revenue

+24%

2.1%

Monthly churn

-0.8 pts

63

Net promoter
score

+11

Onboarding Flow

01

Sign up

Create an
account

02

Connect

Link your
data

03

Configure

Pick your
metrics

04

Launch

Go live in
minutes

2026 Plan

Q1

Ship API

Beta launch

Q2

Mobile app

Integrations

Q3

Enterprise tier

SSO

Sales Pipeline

Visitors · 50,000

Signups · 8,000

Trials · 2,400

Customers · 600

Hierarchy of Needs

Vision

Strategy

Goals

Tactics

SWOT Analysis

Strengths

A strong brand with loyal users

Opportunities

New markets through partnerships

Weaknesses

Thin margins and a small team

Threats

New entrants driving price pressure

Evaluation Matrix

Platform A Platform B Platform C

Security	Strong	Medium	Strong
Workflow	Medium	Strong	Medium
Support	Strong	Strong	Medium

Job Search Funnel (2021)



Final folder shape

```
simple-eda-project/  
  src/  
    simple_eda/  
      __init__.py  
      core.py  
  README.md  
  pyproject.toml  
  LICENSE
```

Agents in AI

How software learned to act, not just answer

What Makes Software an Agent

Perceive

Read its environment.

Reason

Plan a course of action.

Act

Use tools to change things.

Assistant vs Agent

Assistant

Answers one prompt

Waits for you

Has no tools

Agent

Pursues a goal

Acts on its own

Calls tools

Core Capabilities

Planning

Break goals into steps.

Tool Use

Call APIs and code.

Memory

Recall past context.

Reflection

Critique and retry.

Why Now

2022

LLMs Arrive

tool use emerges

10x

Faster

early reports

1M+

Developers

building agents

From Rules to Agents

1980s

Rules

Hand-coded
logic.

2010s

ML

Learned patterns.

Today

Agents

Models that act.

Common Agent Patterns

ReAct

Reason,
then act.

Tool Use

Call
functions.

RAG

Retrieve
facts first.

Multi-Agent

Specialists
team up.

Multi-Agent Systems

Many narrow agents beat one generalist.

Each owns a clear role.

A coordinator routes the work.

They hand off context.

Promise vs Pitfalls

Promise

Automate real work

Scale expertise

Run all day

Pitfalls

Errors compound

Hard to predict

Needs guardrails

Takeaways

Software that acts, with human oversight

Claude Code

A field guide for data scientists

What We Will Cover

- 01 What Claude Code is
- 02 How it differs from chat
- 03 Getting set up
- 04 Everyday data science workflows
- 05 Habits that make it shine



Claude Code

A terminal-based AI agent that reads your project, writes and runs code, inspects the results, and iterates until the job is done.

Chat vs Agent

Chat

You ferry code around

vs

Agent

It works in your repo

What It Can Do in Your Project

Explore

Read files, schemas, and notebooks.

Execute

Run scripts, queries, and tests.

Iterate

Check outputs and fix its own errors.

Set Up in Three Steps

01 Install

`npm install -g @anthropic-ai/claude-code`

02 Open your project

cd into the repo and run claude.

03 Ask for an analysis

Describe the outcome; it plans the code.



a first session

```
$ cd sales-analysis && claude
```

```
> Profile sales.csv: nulls, dtypes,  
  outliers. Save it to profile.md.
```

```
Reading sales.csv (1.2M rows)...
```

```
Running profiling script...
```

```
Wrote profile.md: 3 findings.
```

Everyday Workflows

EDA

Profile new data in minutes.

Cleaning

Fix types, nulls, duplicates.

Modeling

Fast first baselines.

Reporting

Plots and write-ups.

How It Works on a Task

01

Explore

Scans the data and code.

02

Plan

Proposes an approach.

03

Execute

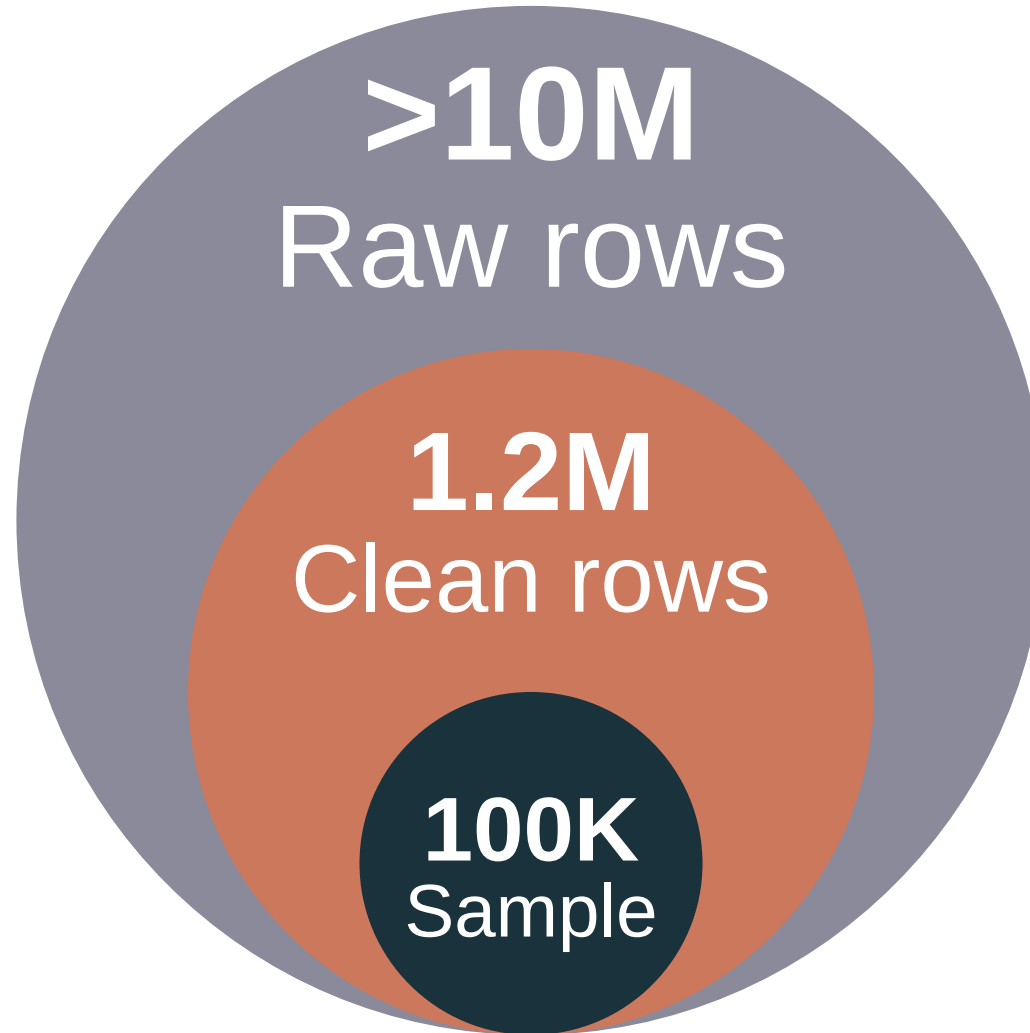
Writes and runs the code.

04

Verify

Checks results, then iterates.

One Prompt, a Whole Pipeline



A Typical Day, Reworked

Before

Boilerplate pandas by hand

Stack Overflow detours

Stale, undocumented
notebooks

With Claude Code

Describe the outcome
instead

Errors debugged in place

Analysis committed with
notes

Commands Worth Knowing

Command

What it does

`claude`

Start a session

`/init`

Write CLAUDE.md

`/compact`

Trim the context

`claude -p`

One-shot mode

Habits That Make It Shine

Keep a CLAUDE.md with data caveats.

Ask for a plan before a big analysis.

Have it verify outputs, not just run.

Commit small, reviewable steps.



**What would you analyze if
the boilerplate wrote itself?**

Start Today

Open a terminal and ask

slidekit First Light

Phase 5 demo: lint-clean, zero screenshots

How slidekit Works

Author IR YAML once

Layout engine solves EMU rects

Linters validate before python-pptx emits

Round-trip test verifies — zero screenshots

Calibration Results

0.076%

Max Width Error

vs LibreOffice render

2%

Error Threshold

hard gate PASS

122

Tests Passing

phases 1 through 5

Design Principles

Deterministic

Same input always produces same output.

No screenshots

Lint errors cite path and exact fix.

Agent-ready

Build and fix loop needs no images.

Before and After

Ad-hoc pptx code

Manual EMU math

Visual QA required

No lint feedback

Agent loops on images

slidekit

Declarative IR YAML

Round-trip verified

Lint errors with fix

Agent loops on text

Build Your First Library

From four functions to a package on PyPI

The path to PyPI

01

Function

pandas
code

02

Package

one
folder

03

Wheel

build a
file

04

PyPI

twine
upload

05

Install

pip install

Today's goals

01 Package

Create a real, importable Python package.

02 Publish

Upload it to TestPyPI, then PyPI.

03 Install

`pip install` your own package.



"Build tiny. Install locally. Publish safely."

- The workshop mantra



What we're building

```
import pandas as pd
import simple_eda as eda

df = pd.read_csv("data.csv")
eda.summarize(df)
eda.missing(df)
```

What we build now — and skip

Skip for now

CI/CD

Testing

Many dependencies

Build now

One dependency: pandas

Plain Python functions

A clear README



01

What is a Python library?

Script vs library

Script

Run: `python analyze.py`

Less convenient

Limited scope

Library

Use: `import simple_eda`

More convenient

Broad scope

Exploratory data analysis

Getting to know a dataset — its rows, columns, types, missing values, and basic stats. Our library inspects data; it does not clean it.

A tiny library is still real

No web app required.

No dashboard required.

No dozens of dependencies.

Just useful, installable, importable.

The five bare minimums

A unique package name

A project folder

An `__init__.py` file

Your functions

Metadata — `pyproject.toml` (Module 5)



02

Design the smallest API



Start from the user's first line

```
import simple_eda as eda  
eda.summarize(df)
```

Pick boring function names

Avoid (clever)

hyper_profile_engine

magic_diagnose

auto_ai_report

mega_summary

Prefer (boring)

summarize

missing

numeric_columns

categorical_columns

One input type at first

Support pandas DataFrames first.

Not Polars, Spark, Dask, or Arrow yet.

Fewer choices means fewer bugs.



Return plain objects

```
{  
  "rows": 100,  
  "columns": 5,  
  "column_names": ["city", "sales"],  
}
```

Never mutate the user's data

Avoid

`df.dropna(inplace=True)`

`df["x"] = ...`

Silent type conversion

Do this

`df.isna().sum()`

`df.describe()`

`list(df.columns)`

The 0.1.0 mindset

First working version, not perfect.

Installable and understandable.

Momentum, not completeness.



03

Create the package folder

Final folder shape (src layout)

```
simple-eda-project/  
  src/  
    simple_eda/  
      __init__.py  
      core.py  
    README.md  
    pyproject.toml  
    LICENSE
```

Install name vs import name

Install name

simple-eda-you

hyphens

used by pip & PyPI

Import name

simple_eda

underscores

used in Python code

Make it in the terminal

```
mkdir simple-eda-project  
cd simple-eda-project  
mkdir -p src/simple_eda  
touch src/simple_eda/__init__.py  
touch src/simple_eda/core.py  
touch README.md pyproject.toml LICENSE
```



`__init__.py`

The file that marks a folder as an importable package and sets its top-level API — it is what lets users write ``import simple_eda``.

Why the src/ layout?

Blocks accidental root imports.

Matches modern Python projects.

Makes packaging behavior clear.

Common folder mistakes

pyproject.toml in the wrong folder.

Forgetting `__init__.py`.

Folder named `simple-eda`, not `simple_eda`.

Installing from the wrong directory.



04

Write the EDA functions



```
summarize()
```

```
def summarize(df):  
    return {  
        "rows": len(df),  
        "columns": len(df.columns),  
        "names": list(df.columns),  
    }
```

Four core functions (+ one optional)

summarize — shape, columns, types

missing — nulls per column

numeric_columns / categorical_columns

stats — optional; pandas already does it



```
missing()
```

```
def missing(df):  
    counts = df.isna().sum()  
    return counts.sort_values(  
        ascending=False)
```

numeric & categorical columns

```
def numeric_columns(df):  
    sel = df.select_dtypes(  
        include="number")  
    return list(sel.columns)
```

```
def categorical_columns(df):  
    sel = df.select_dtypes(  
        include=["object", "category"])  
    return list(sel.columns)
```

Expose the public API

```
# src/simple_eda/__init__.py
from .core import (
    summarize,
    missing,
    numeric_columns,
    categorical_columns,
    stats,
)
# __all__ = [...] lists the public names
```



Try it on a tiny DataFrame

```
import simple_eda as eda
```

```
df = pd.DataFrame({  
    "name": ["Ana", "Bo", None],  
    "age": [25, None, 31],  
})  
print(eda.summarize(df))  
print(eda.missing(df))
```

Function do's and don'ts

Don't

Change df in place

Print instead of return

Add many optional args

One function for everything

Do

Return data

Use plain dicts / Series

Keep signatures small

One job per function



05

Add pyproject.toml

What `pyproject.toml` does

Names your package and version.

Lists dependencies.

Tells build tools how to build.

Replaces old `setup.py` workflows.

A minimal pyproject.toml

```
[build-system]
```

```
requires = ["setuptools>=61"]
```

```
build-backend = "setuptools.build_meta"
```

```
[project]
```

```
name = "simple-eda-yourname"
```

```
version = "0.1.0"
```

```
description = "Tiny EDA helper"
```

```
dependencies = ["pandas>=1.5"]
```

Version numbers

Version

Means

0.1.0

First working version

0.1.1

A small fix

0.2.0

A new feature

1.0.0

Stable and public

README: install & use

simple-eda-yourname

Tiny EDA helper for pandas.

Install

```
pip install simple-eda-yourname
```

Use

```
import simple_eda as eda  
print(eda.summarize(df))
```

License basics

Says what others may do with your code.

MIT is common for small projects.

Use an official template.

Never invent your own license text.



06

Install locally with pip

Install in editable mode

```
# run from the folder with pyproject.toml  
python -m pip install -e .
```

What **-e (editable)** means

Links pip to your source folder.

Code edits apply instantly.

No reinstall while developing.

Smoke-test the import

```
>>> import simple_eda as eda  
>>> eda.__all__  
['summarize', 'missing', ...]
```

When the import fails

Symptom

ModuleNotFoundError

Wrong folder

Wrong environment

Fix

Run `pip install -e .`

`cd` to the project root

Use `python -m pip`



07

Build a real package file

Build the artifacts

```
python -m pip install build
```

```
python -m build
```

The dist/ folder

dist/

■■■ simple_edat_yourname0.1.0.tar.gz

■■■ simple_edat_yourname0.1.0-...whl

Wheel vs source distribution

Source (.tar.gz)

A source archive

Used as a fallback

Wheel (.whl)

Prebuilt, fast install

What most users get

Keep build output out of git

dist/, build/, *.egg-info are generated.

Add them to .gitignore.

Bump the version before rebuilding.



08

Publish safely

Why TestPyPI first?

A practice package index.

Rehearse upload and install commands.

Mistakes are cheap here.

The upload flow

01 Check

```
python -m twine check dist/*
```

02 TestPyPI

```
twine upload --repository testpypi dist/*
```

03 PyPI

```
twine upload dist/* — only after TestPyPI works.
```

Install from TestPyPI

```
# install from TestPyPI
```

```
pip install \  
    -i https://test.pypi.org/simple/ \  
    --extra-index-url \  
        https://pypi.org/simple/ \  
simple-eda-yourname
```

Publishing safety

Use API tokens, not passwords.

Never commit tokens or secrets.

Fewer deps, less attack surface.

Beware typosquatting.



09

Use, improve & share

What to add — and avoid

Avoid early

Full HTML reports

Plugin systems

Every dataframe backend

AI-generated analysis

Good next steps

`quick_report()`

`top_categories()`

`column_summary()`

`duplicate_rows()`



Add a tiny test

```
def test_numeric_columns():  
    df = pd.DataFrame(  
        {"name": ["A"], "age": [10]})  
    cols = eda.numeric_columns(df)  
    assert cols == ["age"]
```

Final checklist

Package files all exist.

`pip install -e .` works.

`python -m build` works.

`twine check dist/*` passes.

README has install + usage.

0.1.0



Ship your first version

Four functions, importable, published.

Interface Aesthetics

Ngo, Teo & Byrne · Information Sciences (2003)



**Is a layout's beauty
subjective — or can it be
computed?**



Interface aesthetics

The visual quality of a layout, written as numbers from where each element sits and how big it is.

14

Aesthetic measures

Closed-form functions of element bounding boxes

The core claim

Layout quality is not purely subjective.

Balance and spacing become numbers.

Each number is a formula over boxes.

Measured, optimised, checked vs people.



01

The fourteen measures

The measures (1–5)

Balance — even visual weight

Equilibrium — mass near the centre

Symmetry — mirrored about the axes

Sequence — a clear reading flow

Cohesion — aspect ratios agree

Bounding box

Every measure reads only position and size — x, y, width, height. No pixels.

The measures (6–10)

Unity — parts read as one

Proportion — pleasing ratios

Simplicity — few alignment points

Density — comfortable coverage

Regularity — elements snap to a grid

The measures (11–14)

Economy — few sizes, reused

Homogeneity — evenly distributed

Rhythm — regular, repeating change

Order & complexity — structure wins

regularity – Ngo (2003)

Ngo regularity (alignment term)

$$\text{regularity} = 1 - (\text{n_vap} + \text{n_hap}) / (2n)$$

a tidy 5-bullet column:

$n = 5$, $\text{n_vap} = 1$, $\text{n_hap} = 5$

$$\text{regularity} = 1 - (1 + 5) / 10 \quad \# 0.40$$

slidekit: left edges only

$$\text{regularity} = 1 - 1 / 5 \quad \# 0.80$$

Same clean column, two scores

0.40

Ngo x + y regularity

vs

0.80

Left-edge alignment

Why slidekit fits this paper

Every element already has an exact rect.

Measures are arithmetic over geometry.

Deterministic — same deck, same score.

Zero rendering, zero vision.



02

From paper to slidekit

What slidekit adopted

Declined

Regularity

Rhythm

Proportion

Economy

Spacing-regularity

Adopted

Balance

Density

Adoption decisions

Measure	Verdict	Reason
Balance	Adopted	Axis moment
Density	Adopted	Coverage
Regularity	Declined	Halves cols
Proportion	Declined	Ratio noise

A measured-aesthetics lineage

1933

Birkhoff

A beauty
ratio

2003

Ngo

Fourteen
measures

2004

Lok

Visual
balance

2026

Zhang

Moment
model

Honest scope

The journal paper is paywalled.

Public copies reproduce only alignment.

Constants are not copied blindly.

No universal beauty equation is claimed.

From measure to code

01 Read geometry

Take each element's rect.

02 Define a formula

Write an open, closed-form score.

03 Verify & ship

Check the source, then report.



**"Turn design quality into explicit,
auditable measures."**

- Ngo, Teo & Byrne (2003)